

PDF to Audiobook

Deep Dive: How the Code Works, Line by Line

Explainer document generated from source

Contents

1	What this project is	2
2	Architecture overview	3
3	converter.py: the conversion logic	4
3.1	extract_text_from_pdf(pdf_path)	4
3.2	convert_pdf_to_audio(pdf_path, output_dir=None, lang="en")	4
3.3	The command-line interface	6
4	main.py: the Tkinter GUI	7
4.1	Imports and global state	7
4.2	select_files() and add_files()	7
4.3	convert_pdf_to_tts(): the GUI/converter boundary	8
4.4	Building the window	9
5	Sequence: a full conversion, end to end	10
6	Directory layout and output	11
7	Dependencies	11
8	Error handling, end to end	11
9	Licensing note	12
10	Summary	12

1 What this project is

PDF to Audiobook is a small desktop application, written in Python, that turns PDF documents into audio files. A user picks one or more PDF files through a window with buttons, clicks “Convert,” and for each PDF gets back an MP3 file that reads the document’s text out loud.

The whole project is two source files:

- **converter.py** — the part that does the actual work: pulling text out of a PDF and turning that text into speech. It has no dependency on any window or button; it is plain functions that take a file path and return a result. It also doubles as a command-line tool.
- **main.py** — the graphical window (built with a toolkit called Tkinter) that lets a human click buttons instead of typing commands. It does not know how to extract text or make speech itself; it calls into **converter.py** to do that.

This split — “logic” in one file, “window and buttons” in another — is the central architectural decision in this project, and this document explains both halves and exactly how they talk to each other.

Beginner terms used throughout this document

GUI (Graphical User Interface) A program you interact with using windows, buttons, and a mouse, instead of typing text commands into a terminal.

GUI toolkit A programming library that provides the building blocks (windows, buttons, labels, text fields) used to construct a GUI. This project uses **Tkinter**, which ships with Python itself, so no extra installation is needed just for the window to appear.

TTS (Text-to-Speech) Software that converts written text into spoken audio. Here it is provided by a library called **gTTS** (Google Text-to-Speech), which is a thin wrapper around Google Translate’s public speech-synthesis web service.

PDF text extraction The process of reading the words out of a PDF file. PDFs are primarily a *visual* format (they describe where ink should go on a page), so “getting the text back out” is a distinct, sometimes lossy step, handled here by a library called **PyPDF2**.

CLI (Command-Line Interface) A way of running a program by typing a command with arguments into a terminal, instead of clicking buttons in a window.

Module A single Python source file, which can define functions and be imported by other files. **converter.py** and **main.py** are each a module.

Event loop Code that sits in a loop waiting for things to happen (a button click, a key press) and calls the appropriate function when they do. Tkinter’s **root.mainloop()** is one; it is what keeps the window open and responsive.

2 Architecture overview

Figure 1 shows every moving part of the project and how data flows between them: a PDF file goes in, an MP3 file comes out, and two external services (neither of which is bundled with the project; both are installed as Python packages) do the heavy lifting in between.

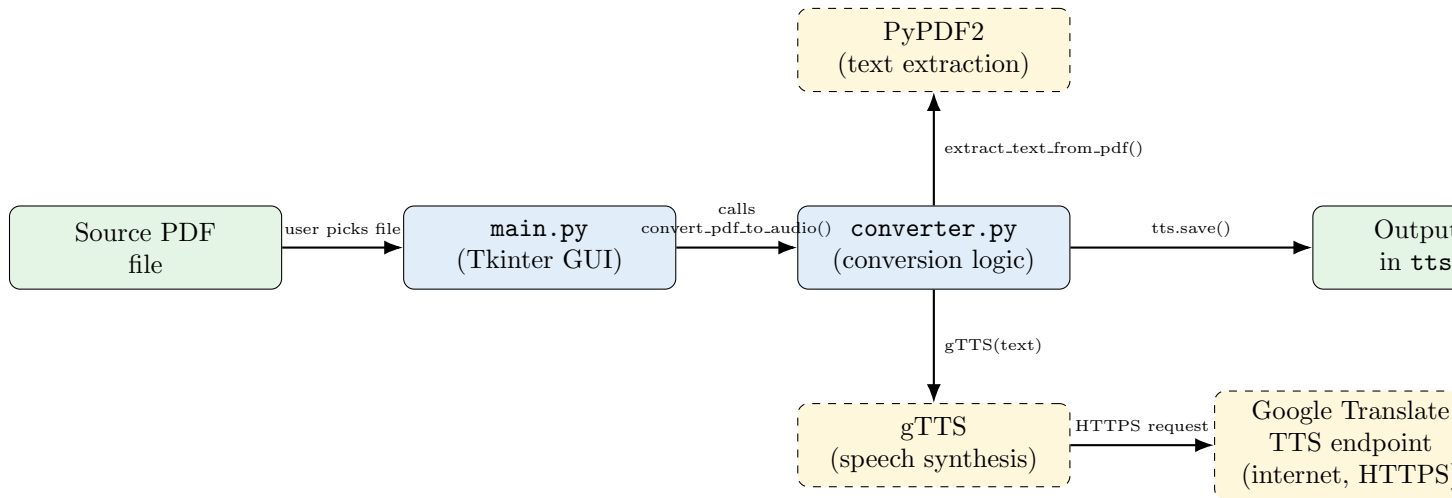


Figure 1: Data flow through the project. `main.py` never touches `PyPDF2` or `gTTS` directly — it only ever calls the two public functions exposed by `converter.py`.

The command-line path (running `converter.py` directly, described in Section 3.3) skips `main.py` entirely and calls the same conversion function from an `argparse`-based entry point instead of from button clicks.

3 converter.py: the conversion logic

This file contains all the code that actually reads a PDF and produces an MP3. It is 16 lines of actual logic plus a small CLI wrapper, and it imports nothing from Tkinter — meaning it can run on a server with no screen attached, and can be imported and called from a test script or another program without ever opening a window.

3.1 extract_text_from_pdf(pdf_path)

```

1 def extract_text_from_pdf(pdf_path: str) -> str:
2     """Extract and concatenate the text of every page in a PDF."""
3     with open(pdf_path, "rb") as f:
4         reader = PdfReader(f)
5         pages_text = []
6         for page in reader.pages:
7             page_text = page.extract_text() or ""
8             pages_text.append(page_text)
9     return "\n".join(pages_text)

```

Walking through this line by line:

- `open(pdf_path, "rb")` opens the file in **binary** mode ("rb" = read, binary). PDFs are not plain text files — they contain a mix of binary structures, fonts, and encoded streams — so they must be opened as raw bytes, not as text.
- The `with` statement is a **context manager**: it guarantees the file handle is closed automatically once the indented block finishes, even if an error is raised inside it.
- `PdfReader(f)` (from the PyPDF2 library) parses the binary PDF structure and exposes it as a Python object with a `.pages` list.
- The loop visits every page and calls `page.extract_text()`, which returns the text PyPDF2 could recover from that page. Text extraction from a PDF is not always reliable: some PDFs are scans (pictures of pages, with no real text layer) or use exotic fonts/encodings that confuse extraction. In these cases PyPDF2's `extract_text()` can return `None` instead of raising an error.
- `page.extract_text() or ""` is a defensive fallback: if `extract_text()` returns `None` (a falsy value in Python), the `or` operator substitutes an empty string instead. Without this, appending `None` into a list that later gets joined with `"\n".join(...)` would raise a `TypeError`, because `join` requires every item to be a string. This is exactly the kind of one-line defensive fix that prevents a single bad page from crashing the extraction of an entire book.
- Finally, `"\n".join(pages_text)` concatenates every page's text into one long string, with a newline character between pages so page boundaries are not run together mid-sentence.

3.2 convert_pdf_to_audio(pdf_path, output_dir=None, lang="en")

```

1 def convert_pdf_to_audio(pdf_path: str, output_dir: str = None, lang: str = "en")
2     -> str:
3     """Extract the text from 'pdf_path' and save it as an mp3.
4
5     If 'output_dir' is not given, a 'tts_pdf' folder is created next to the
6     source PDF. Returns the path to the saved mp3 file.
7     """

```

```

7     text = extract_text_from_pdf(pdf_path)
8     if not text.strip():
9         raise ValueError(f"No extractable text found in '{pdf_path}'")
10
11     pdf_dir = os.path.dirname(os.path.abspath(pdf_path))
12     if output_dir is None:
13         output_dir = os.path.join(pdf_dir, "tts_pdf")
14     os.makedirs(output_dir, exist_ok=True)
15
16     base_name = os.path.splitext(os.path.basename(pdf_path))[0]
17     save_path = os.path.join(output_dir, f"{base_name}_tts.mp3")
18
19     tts = gTTS(text=text, lang=lang)
20     tts.save(save_path)
21     return save_path

```

This is the single function both the GUI and the CLI call to do a full conversion. Step by step:

1. **Extract text** by calling the function above.
2. **Guard against empty PDFs.** `text.strip()` removes leading and trailing whitespace; if what remains is empty (an image-only PDF that yielded no text at all, for instance), the function raises a `ValueError` with a message naming the offending file, rather than silently producing an empty or broken MP3. This is a deliberate “fail loudly and specifically” choice.
3. **Work out where to save the file.** `os.path.abspath(pdf_path)` turns a possibly relative path (like “report.pdf”) into a full, unambiguous path (like “/home/user/docs/report.pdf”). `os.path.dirname(...)` then strips the filename off, leaving just the containing folder. If the caller did not specify an `output_dir`, a subfolder named `tts_pdf` is used, placed inside that same folder — so audio files always land next to the PDF they came from unless told otherwise.
4. `os.makedirs(output_dir, exist_ok=True)` creates that output folder (and any missing parent folders) if it does not already exist. `exist_ok=True` means it does *not* raise an error if the folder is already there — this one call safely handles both “folder is missing, create it” and “folder already exists, do nothing” with no separate existence check needed beforehand.
5. **Build the output filename.** `os.path.splitext(os.path.basename(pdf_path))[0]` takes just the filename (no folder) and drops its extension, so “report.pdf” becomes “report”. That base name gets “_tts.mp3” appended, giving (for example) `report_tts.mp3`.
6. **Synthesize speech.** `gTTS(text=text, lang=lang)` constructs a `gTTS` object holding the full extracted text and a language code (English, “en”, by default). `tts.save(save_path)` is where the actual network request happens: `gTTS` sends the text to Google Translate’s public text-to-speech endpoint over HTTPS and writes the returned audio stream to disk as an MP3 file at `save_path`.
7. The function returns the path it just saved to, so callers (the GUI or the CLI) can report it back to the user.

Important limitation, stated plainly. `gTTS` is not a local/offline TTS engine — it is a network client for a remote Google service. **This means the whole program requires a working internet connection.** There is no local fallback voice, and no API key is required, but the tool is entirely dependent on that external endpoint being reachable and not rate-limiting the request. If the network is down, `tts.save()` raises

a networking exception, which is caught generically (see Section 4.3) — the failure is visible in the console and, in the GUI, only as part of an aggregate “conversion failed” count, with no message distinguishing “no internet” from “bad PDF.”

3.3 The command-line interface

```
1 if __name__ == "__main__":
2     import argparse
3
4     parser = argparse.ArgumentParser(
5         description="Convert a PDF's text to an mp3 audiobook (command line, no
6             GUI)."
7     )
8     parser.add_argument("pdf", help="Path to the source PDF file")
9     parser.add_argument(
10         "-o", "--output-dir", default=None,
11         help="Directory to save the mp3 in (default: <pdf folder>/tts_pdf)"
12     )
13     parser.add_argument(
14         "-l", "--lang", default="en",
15         help="gTTS language code (default: en)"
16     )
17     args = parser.parse_args()
18
19     out_path = convert_pdf_to_audio(args.pdf, args.output_dir, args.lang)
20     print(f"Saved: {out_path}")
```

`if __name__ == "__main__":` is a standard Python idiom meaning “only run the code below when this file is executed directly (`python3 converter.py ...`), not when it is imported by another file (like `main.py` does).” This is precisely what lets `main.py` import `convert_pdf_to_audio` without also triggering an attempt to parse command-line arguments meant for a different invocation.

`argparse` is Python’s standard library for building command-line tools: it turns a description of expected arguments (a required PDF path, an optional output directory, an optional language code) into automatic `--help` text and argument parsing, with no manual string-splitting of `sys.argv` required.

Running `python3 converter.py report.pdf` therefore performs a full, GUI-free conversion, printing the saved path on success or crashing with a Python traceback for the caller to see on failure (unlike the GUI, which catches errors and keeps running — see below).

4 main.py: the Tkinter GUI

This file builds and runs the window a user actually interacts with. It has three buttons (Select Files, Add Files, Convert), a status label, and roughly ten lines of instructional text; underneath, it holds one piece of state — a list of chosen file paths — and delegates all real work to `converter.py`.

4.1 Imports and global state

```
1 import tkinter as tk
2 from tkinter import filedialog
3
4 from converter import convert_pdf_to_audio
5
6 filenames = []
```

`tkinter` is Python’s built-in GUI toolkit (no separate installation needed). `filedialog` is a Tkinter submodule that provides native “Open File” dialog boxes, so the file picker looks like the operating system’s own file browser rather than a custom-drawn one.

`filenames = []` is a **module-level global variable**: a single list, shared by every function in the file, that holds the paths of PDFs the user has selected so far but not yet converted. Because Tkinter button callbacks take no arguments and return no value (they are just called when a click happens), the functions below use `global filenames` to read and modify this shared list instead of passing it around as a parameter — a common, if not the most modern, pattern in small Tkinter scripts.

4.2 `select_files()` and `add_files()`

```
1 def select_files():
2     global filenames
3     f_types = [('PDF files', '*.pdf')]
4     filenames = list(filedialog.askopenfilenames(filetypes=f_types))
5     if len(filenames) == 0:
6         print("No files selected yet!")
7         error_message.config(text="No files selected yet!", fg="red")
8     else:
9         error_message.config(text="Files selected successfully!", fg="green")
```

`filedialog.askopenfilenames(...)` opens the operating system’s native “choose files” window, restricted by `f_types` to show only `.pdf` files, and lets the user pick one or several at once (multi-select). It returns a tuple of the chosen paths (or an empty tuple if the user cancels), which is converted to a list and stored, **replacing** whatever was in `filenames` before.

`error_message.config(text=..., fg=...)` updates the status label’s text and foreground (font) color — green for success, red for a problem — giving the user immediate visual feedback without a popup dialog.

`add_files()` is nearly identical but uses `filenames.extend(...)` instead of replacing the list, so a user can build up a batch across multiple picker invocations (e.g. select files from one folder, then add more from another) instead of starting over each time.

4.3 convert_pdf_to_tts(): the GUI/converter boundary

This is the function that actually bridges the GUI and the conversion logic, and the most important one to understand for how the two halves of the project interact.

```

1 def convert_pdf_to_tts():
2     global filenames
3     if len(filenames) == 0:
4         print("No files selected yet!")
5         error_message.config(text="No files selected yet!", fg="red")
6         return
7
8     error_message.config(text="Converting, please wait...", fg="black")
9     root.update_idletasks()
10
11     converted = 0
12     for filename in filenames:
13         try:
14             save_path = convert_pdf_to_audio(filename)
15             print(f"Saved: {save_path}")
16             converted += 1
17         except Exception as exc:
18             print(f"Failed to convert '{filename}': {exc}")
19
20     filenames = []
21
22     if converted:
23         error_message.config(text=f"Converted {converted} file(s) successfully!",
24                               fg="green")
25     else:
26         error_message.config(text="Conversion failed, see console for details.",
27                               fg="red")

```

1. If no files have been chosen, it bails out immediately with a red status message and does nothing further.
2. It sets the status label to “Converting, please wait...” and calls `root.update_idletasks()`. This forces Tkinter to redraw the window *right now*, before the (potentially slow) conversion loop starts — without this call, Tkinter would not repaint the label until it got back to its own event loop, so the “please wait” message might never actually appear on screen before the program looked frozen.
3. It then loops over every selected file path and, for each one, calls `convert_pdf_to_audio(filename)` — the single function exported from `converter.py` — with default output directory and language. This is the entire interface between the two modules: one function call, one string argument in, one string (the saved path) or an exception out.
4. Each call is wrapped in its own `try/except Exception`. This means a single bad file (corrupted PDF, image-only PDF with no text, a network hiccup during TTS) does not stop the batch — the loop moves on to the next file, and the failure is only printed to the console (visible if the program was started from a terminal, invisible otherwise) rather than shown in the window.
5. After the loop, `filenames` is reset to an empty list, clearing the batch regardless of how many conversions succeeded, and the status label reports an aggregate count: how many succeeded in total, with no per-file detail if some failed and some didn’t.

What this boundary does *not* do. There is no threading here: `convert_pdf_to_tts` runs entirely on Tkinter's own main thread, which is also the thread running the window's event loop. Because `convert_pdf_to_audio` makes a real network request per file (via gTTS) and this can take anywhere from under a second to several seconds depending on document length and network conditions, the window is **unresponsive** for the whole duration of a conversion batch — it cannot be moved, resized, or clicked until the loop finishes. The single `root.update_idletasks()` call before the loop starts only forces one immediate repaint; it does not keep the window responsive *during* the loop.

4.4 Building the window

The remainder of `main.py` is declarative widget setup: it creates the main window (`tk.Tk()`), sets its title, fixed size (900x650, and non-resizable), and background color, then places a title label, a description label, an instructions label, three buttons, and a status label at explicit relative coordinates using `.place(relx=..., rely=..., anchor="center")` — a Tkinter geometry manager that positions widgets as a fraction of the window's width/height (e.g. `relx=0.5` is horizontally centered) rather than using a grid or box-packing layout. Each button's `command=` argument wires it to one of the three functions above:

Button	Calls
Select Files	<code>select_files()</code>
Add Files	<code>add_files()</code>
Convert	<code>convert_pdf_to_tts()</code>

Finally:

```
1 if __name__ == "__main__":
2     root.mainloop()
```

`root.mainloop()` starts Tkinter's event loop, which blocks (keeps running) until the window is closed, continuously waiting for and dispatching events like button clicks to the functions registered above. Note that the window *creation* code above this guard (creating `root`, the labels, and the buttons) runs unconditionally at import time, but the loop that actually displays and keeps the window alive only starts when the file is run directly — consistent with the same `__name__ == "__main__"` pattern used in `converter.py`.

5 Sequence: a full conversion, end to end

Figure 2 traces exactly what happens, function by function, when a user opens the app, picks two PDFs, and clicks Convert.

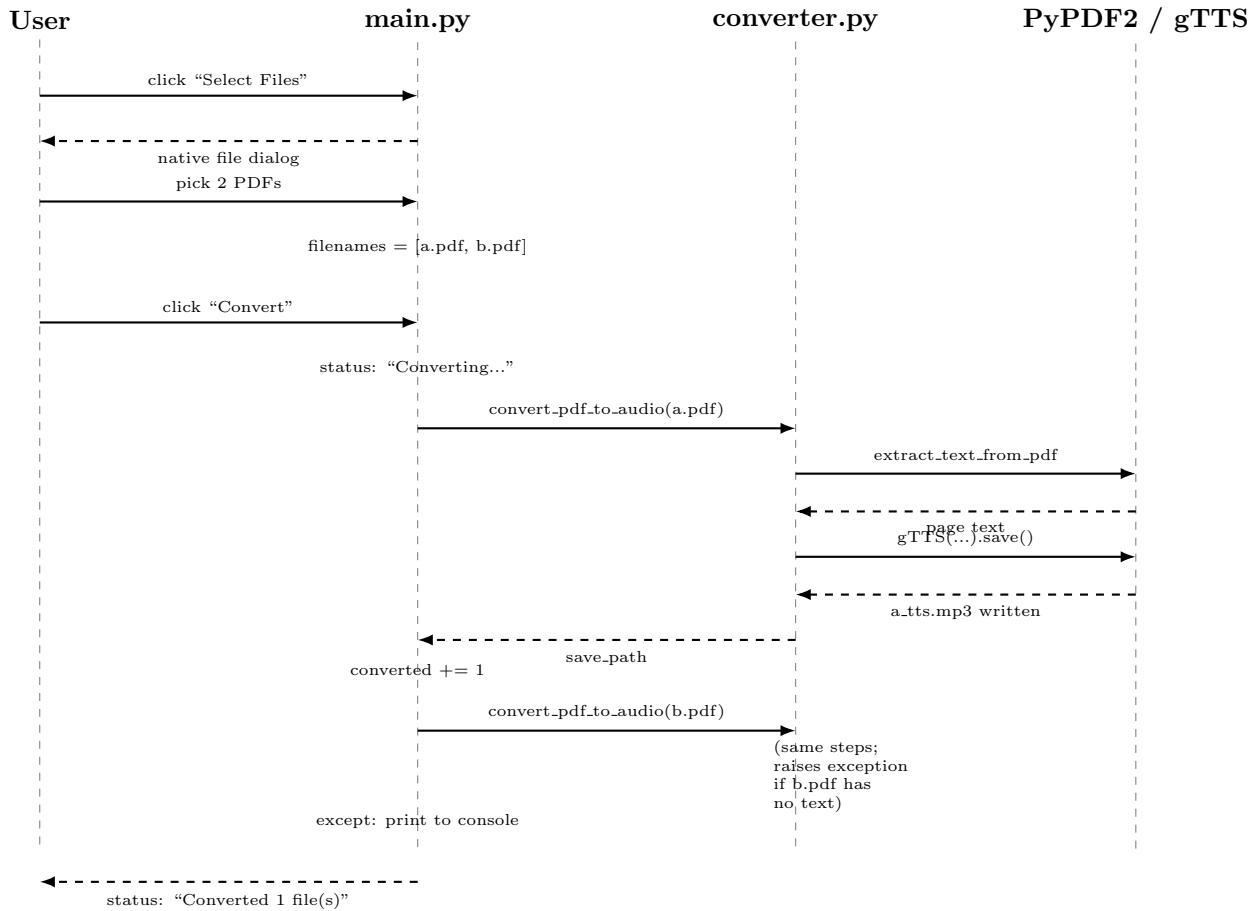


Figure 2: Sequence for converting a batch of two PDFs, one of which fails. Note that every step runs sequentially on the same thread as the window itself.

6 Directory layout and output

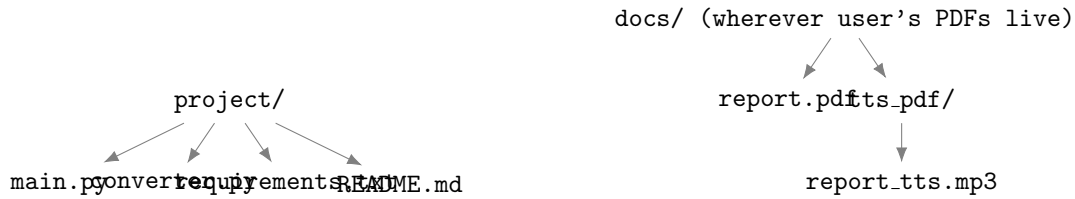


Figure 3: Left: the project’s own source files. Right: where output lands — a `tts.pdf` folder created *next to* the source PDF, not inside the project. This folder can be anywhere on disk; the project does not require PDFs to live inside it.

7 Dependencies

`requirements.txt` pins exactly two third-party packages:

Package	Role
PyPDF2==3.0.1	Parses PDF files and extracts text (Section 3.1).
gTTS==2.5.4	Sends text to Google’s TTS endpoint and saves the MP3 result.

Tkinter is not listed because it ships as part of the Python standard library on most platforms and is not installed via `pip`. `argparse` and `os` are also standard library modules, requiring no installation.

The project reads no environment variables (documented explicitly in `.env.example`) and requires no API key: gTTS calls a public, unauthenticated Google Translate endpoint.

8 Error handling, end to end

Failure	Where it’s caught	What the user sees
Page has no extractable text	Handled inline (or “”)	Nothing — silently treated as an empty page
Whole PDF has no extractable text	<code>ValueError</code> raised in <code>convert_pdf_to_audio</code>	In the GUI: swallowed by the batch’s generic <code>except</code> , only visible via the aggregate failure count and a console line. On the CLI: an uncaught traceback.
Network/gTTS failure	Not caught in <code>converter.py</code>	Same as above: GUI swallows it per-file; CLI shows a traceback.
No files selected	Explicit check in <code>convert_pdf_to_tts</code>	Clear red status message, no exception involved.

This table makes visible a real, documented limitation of the current design: the GUI cannot distinguish “this PDF has no text” from “the network is down” from any other exception, because

`convert_pdf_to_tts` catches the broad `Exception` class per file and only logs the message to the console, which most users running the packaged app will never see. The project’s own README states this plainly under “Challenges” and “What I’d do differently” rather than overstating what the current error handling does.

9 Licensing note

The project ships under the PolyForm Strict License 1.0.0, which permits personal, noncommercial, and specific noncommercial-organization use, but does not grant a license to redistribute the software or distribute modified versions of it. This is a project-level packaging detail, not a code-architecture one, but it is included here for completeness since it governs what a reader of this document is actually allowed to do with the code.

10 Summary

`converter.py` and `main.py` form a clean, minimal split: all PDF-reading and speech-synthesis logic lives in one GUI-free module built around a single public function, `convert_pdf_to_audio`, and the Tkinter window in the other module does nothing but collect file paths from the user and call that one function once per file, translating its return value or exception into a status-label update. The seam between the two is exactly one function call wide, which is what lets `converter.py` be run, tested, and reasoned about entirely independently of any window ever being opened.